

# Deep Learning

## Descida de Gradiente

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory  
PUCRS

junho de 2026

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
8. Encerramento

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
8. Encerramento

## Um exemplo concreto: regressão logística com 1 peso e 1 bias

- Uma única unidade sigmoide com 2 parâmetros (peso  $w$  e bias  $b$ ).
- Função de custo: erro quadrático médio.

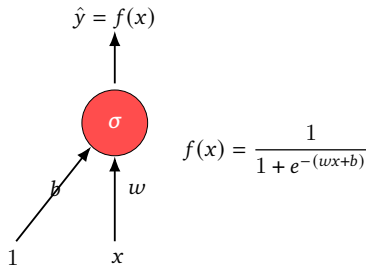
$$\mathcal{L}(w, b) = \frac{1}{N} \sum_{i=1}^N \left( y^{(i)} - f(x^{(i)}) \right)^2$$

- Pergunta natural: por que não simplesmente

$$\operatorname{argmin}_{w, b} \mathcal{L}(w, b)$$

via varredura por força bruta?

- [Vamos tentar](#) (visualização interativa, IITM CS6910)





## E se tentássemos varrer todos os $(w, b)$ ?

---

- Com 2 parâmetros conseguimos visualizar a paisagem da loss em uma grade.
- Para 2 parâmetros, com  $k$  valores em cada eixo, avaliamos  $k^2$  pontos.
- Para uma rede com  $P$  parâmetros, avaliamos  $k^P$  pontos.

### Para fixar a ideia

Mesmo um MLP modesto com  $P=1\,000$  parâmetros e apenas  $k=10$  valores por eixo exigiria  $10^{1000}$  avaliações. Inviável.

## Precisamos de um método guiado

---

- Força bruta ignora qualquer estrutura da função.
- Em problemas suaves (diferenciáveis), a própria função carrega informação local sobre como reduzir a loss.
- A descida de gradiente é justamente um método que usa essa informação local para escolher a direção e o tamanho do próximo passo.

# Visão Geral

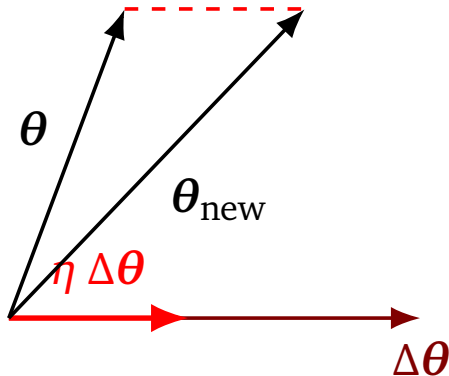
---

1. Por que não força bruta
- 2. Derivando a Descida de Gradiente**
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
8. Encerramento

## Estratégia: aplicar uma atualização que reduza a loss

---

- Queremos achar uma atualização  $\Delta\theta$  tal que  $\mathcal{L}(\theta + \eta\Delta\theta) < \mathcal{L}(\theta)$ .
- $\eta > 0$  é a *taxa de aprendizado* (*learning rate*).
- A pergunta é: **qual direção  $\Delta\theta$  escolher?**

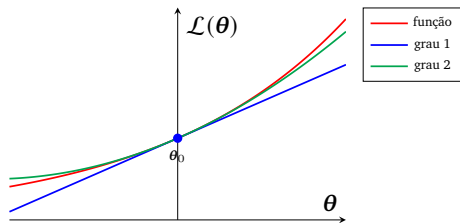


# Refresher: Série de Taylor<sup>1</sup>

---

- Aproxima localmente uma função suave  $\mathcal{L}(\theta)$  por um polinômio em torno de  $\theta_0$ .
- Quanto mais termos, melhor a aproximação.
- Para  $\eta$  pequeno, a primeira ordem domina.

$$\mathcal{L}(\theta) = \mathcal{L}(\theta_0) + \frac{\nabla \mathcal{L}(\theta_0)}{1!}(\theta - \theta_0) + \frac{\nabla^2 \mathcal{L}(\theta_0)}{2!}(\theta - \theta_0)^2 + \dots$$



---

<sup>1</sup>Visualização interativa: IITM CS6910 (Taylor).

## Aplicando Taylor à Loss

---

Considere  $\Delta\theta = u$  para simplificar a notação. Em torno de  $\theta$ :

$$\mathcal{L}(\theta + \eta u) = \mathcal{L}(\theta) + \eta u^\top \nabla_{\theta} \mathcal{L}(\theta) + \frac{\eta^2}{2!} u^\top \nabla_{\theta}^2 \mathcal{L}(\theta) u + \dots$$

- Para  $\eta$  suficientemente pequeno,  $\eta^2, \eta^3, \dots$  são desprezíveis.
- Ficamos com a aproximação de primeira ordem:

$$\mathcal{L}(\theta + \eta u) \approx \mathcal{L}(\theta) + \eta u^\top \nabla_{\theta} \mathcal{L}(\theta).$$

## Critério de melhoria

---

Queremos que a atualização reduza a loss, ou seja

$$\mathcal{L}(\boldsymbol{\theta} + \eta u) - \mathcal{L}(\boldsymbol{\theta}) < 0.$$

Substituindo a aproximação de primeira ordem:

$$\eta u^\top \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) < 0.$$

- Como  $\eta > 0$ , basta exigir  $u^\top \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) < 0$ .
- Isto é, o produto escalar entre a direção de atualização e o gradiente deve ser **negativo**.

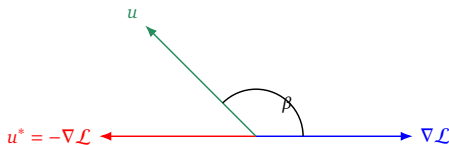
## Quando o produto escalar é negativo?

---

Seja  $\beta$  o ângulo entre  $u$  e  $\nabla_{\theta}\mathcal{L}(\theta)$ .

$$u^{\top}\nabla_{\theta}\mathcal{L}(\theta) = \|u\| \|\nabla_{\theta}\mathcal{L}(\theta)\| \cos(\beta).$$

- $\|u\| \geq 0$  e  $\|\nabla\mathcal{L}\| \geq 0$  sempre.
- Logo, o sinal vem inteiramente de  $\cos(\beta)$ .
- Negativo para  $\beta \in (90^{\circ}, 180^{\circ})$ .
- **Mais negativo** para  $\beta = 180^{\circ}$ , ou seja  $u = -\nabla_{\theta}\mathcal{L}$ .





# Regra de atualização da Descida de Gradiente

---

## Direção ótima

A escolha que minimiza o produto escalar é  $u = -\nabla_{\theta} \mathcal{L}(\theta)$ .

Substituindo na atualização:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} \mathcal{L}(\theta_t)$$

- Cada passo move os parâmetros na direção oposta ao gradiente.
- $\eta$  controla o tamanho do passo.
- Esta é a **descida de gradiente**<sup>2</sup>.

---

<sup>2</sup>Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.

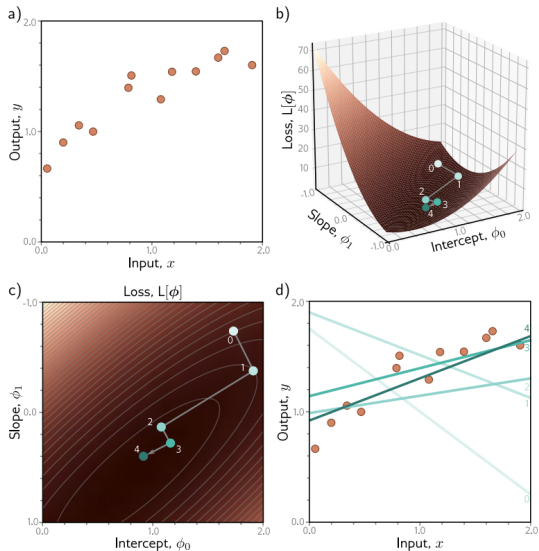
# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
- 3. Convexidade e Paisagem da Loss**
4. Tamanho do Mini-batch
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
8. Encerramento

# Caso 1: função convexa

- Modelo linear:  $\hat{y} = \theta_0 + \theta_1 x$ .
- Loss MSE:  $\mathcal{L}(\theta) = (y - \hat{y})^\top (y - \hat{y})$ .
- A superfície tem um único mínimo.
- Qualquer inicialização converge ao mínimo **global**, desde que  $\eta$  seja suficientemente pequeno.

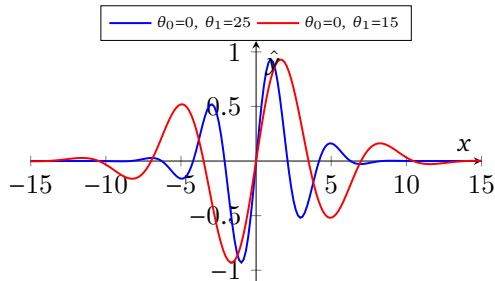


## Caso 2: função não convexa, modelo de Gabor

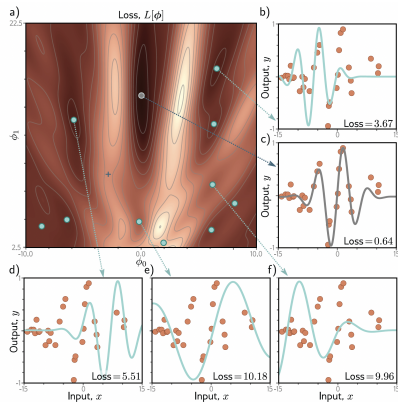
Modelo com apenas 2 parâmetros:

$$\hat{y} = \sin(\theta_0 + 0.06 \theta_1 x) \exp\left(-\frac{(\theta_0 + 0.06 \theta_1 x)^2}{32}\right)$$

- $\theta_0$  controla o deslocamento.
- $\theta_1$  controla a frequência.
- Apesar de ter só 2 parâmetros, ajustar a curva aos dados já é não trivial.

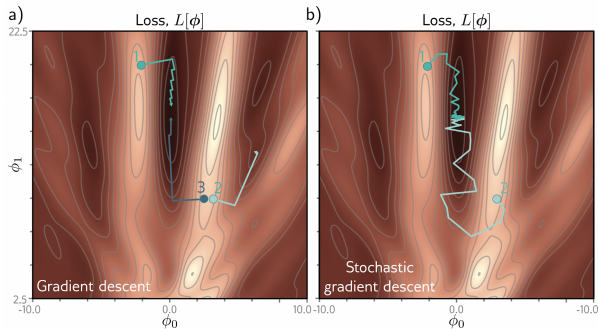


# Paisagem da loss do modelo de Gabor



**Figure 6.4** Loss function for the Gabor model. a) The loss function is non-convex, with multiple local minima (cyan circles) in addition to the global minimum (gray circle). It also contains saddle points where the gradient is locally zero, but the function increases in one direction and decreases in the other. The blue cross is an example of a saddle point; the function decreases as we move horizontally in either direction but increases as we move vertically. b-f) Models associated with the different minima. In each case, there is no small change that decreases the loss. Panel (c) shows the global minimum, which has a loss of 0.64.

# Descida em uma loss não convexa



**Figure 6.5** Gradient descent vs. stochastic gradient descent. a) Gradient descent with line search. As long as the gradient descent algorithm is initialized in the right “valley” of the loss function (e.g., points 1 and 3), the parameter estimate will move steadily toward the global minimum. However, if it is initialized outside this valley (e.g., point 2), it will descend toward one of the local minima. b) Stochastic gradient descent adds noise to the optimization process, so it is possible to escape from the wrong valley (e.g., point 2) and still reach the global minimum.

- Para inicializações em “bons vales”, a descida converge a um mínimo **local** próximo ao global.
- Para inicializações em vales rasos, a descida fica presa.
- *Spoiler*: em redes neurais profundas, a paisagem **nunca** é convexa.
- Precisamos de variações que ajudem a sair de pontos ruins.

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
- 4. Tamanho do Mini-batch**
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
8. Encerramento

## Variar quantas instâncias entram no gradiente

---

- Podemos estimar o gradiente com diferentes tamanhos de *batch*:

| Tamanho do batch     | Nome usual                               | Atualização           |
|----------------------|------------------------------------------|-----------------------|
| $N$ (todo o dataset) | <i>Batch Gradient Descent</i>            | determinística        |
| $1 < m < N$          | <i>Mini-batch Gradient Descent</i>       | estocástica           |
| 1                    | <i>Stochastic Gradient Descent</i> (SGD) | altamente estocástica |

- Em deep learning, “SGD” costuma ser usado como sinônimo de mini-batch com  $m \ll N$ .



## Por que não usar sempre o batch completo?

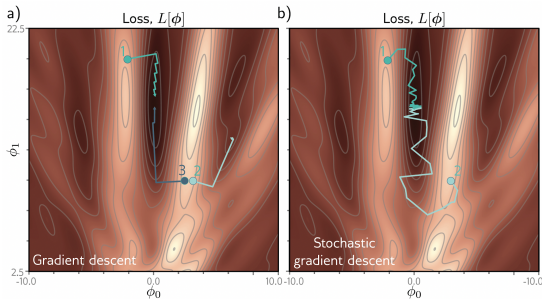
---

- Limitações de *hardware*: o batch precisa caber na memória da GPU.
- Custo por passo: cada atualização vê  $N$  exemplos, mesmo quando uma fração já seria suficiente para uma boa estimativa do gradiente.
- Falta de ruído: a trajetória é determinística e não consegue escapar de mínimos locais ou pontos de sela.

### Trade-off

Mini-batches pequenos introduzem ruído (bom para escapar) e cabem em memória, mas a estimativa do gradiente é mais barulhenta (passos mais erráticos).

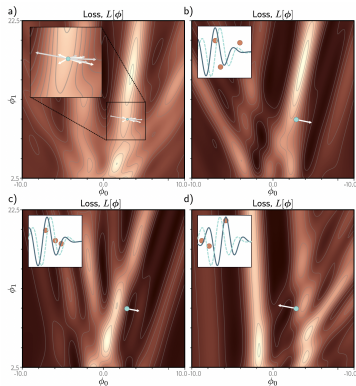
# Trajetória: *Batch* vs *Stochastic*



**Figure 6.5** Gradient descent vs. stochastic gradient descent. a) Gradient descent with line search. As long as the gradient descent algorithm is initialized in the right “valley” of the loss function (e.g., points 1 and 3), the parameter estimate will move steadily toward the global minimum. However, if it is initialized outside this valley (e.g., point 2), it will descend toward one of the local minima. b) Stochastic gradient descent adds noise to the optimization process, so it is possible to escape from the wrong valley (e.g., point 2) and still reach the global minimum.

- Esquerda: *batch gradient descent* usa o gradiente exato em cada passo.
- Direita: *stochastic gradient descent* oscila, mas explora vales que o batch ignora.

# Mini-batches em uma paisagem real



**Figure 6.6** Alternative view of SGD for the Gabor model with a batch size of three. a) Loss function for the entire training dataset. At each iteration, there is a probability distribution of possible parameter changes (inset shows samples). These correspond to different choices of the three batch elements. b) Loss function for one possible batch. The SGD algorithm moves in the downhill direction on this function for a distance that is determined by the learning rate and the local gradient magnitude. The current model (dashed function in inset) changes to better fit the batch data (solid function). c) A different batch creates a different loss function and results in a different update. d) For this batch, the algorithm moves *downhill* with respect to the batch loss function but *uphill* with respect to the global loss function in panel (a). This is how SGD can escape local minima.

- Em a) está a loss completa.
- Em b), c), d) vemos diferentes mini-batches com  $m=3$ .
- Cada mini-batch aproxima o gradiente verdadeiro com erro, o que se traduz em ruído na trajetória.

# Propriedades do SGD

---

- Instâncias amostradas **sem reposição**; uma **época** é completada quando todas as instâncias foram vistas.
- Adiciona ruído ao treinamento, ajudando a escapar de mínimos locais e pontos de sela.
- É computacionalmente leve (uma instância por passo).
- Empiricamente, soluções obtidas com SGD generalizam melhor.
- Não converge a um ponto fixo (passos sempre flutuam ao redor do mínimo).

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
- 5. Momentum e Nesterov**
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
8. Encerramento

## Motivação: usar a memória dos gradientes

---

- Gradientes consecutivos podem oscilar fortemente (mini-batch ruidoso ou vales estreitos).
- Ideia: somar uma fração da atualização anterior à nova, suavizando a trajetória.
- Se os gradientes recentes apontam na mesma direção, aceleramos.
- Se mudam de direção, o efeito tende a se cancelar e o passo é amortecido.

## Atualização com momentum

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla_{\theta} \mathcal{L}(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_{t+1}$$

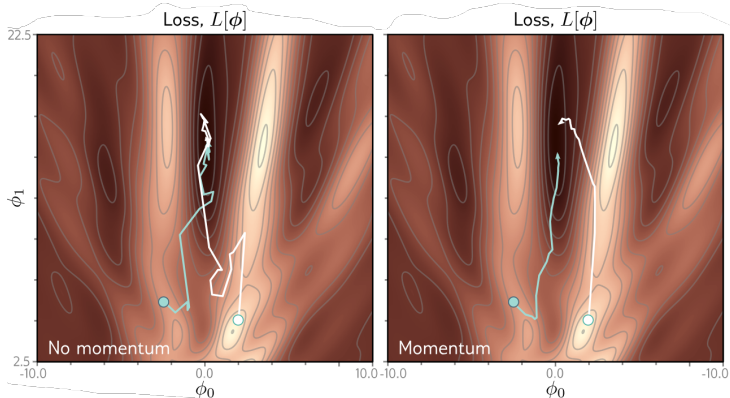
- $\beta \in [0, 1)$  controla quanto da memória é considerado.
- $v_t$  é uma **média móvel exponencial** dos gradientes anteriores.
- Para  $\beta = 0$ , recuperamos a descida de gradiente padrão.
- Valores típicos:  $\beta \in [0.9; 0.99]$ .

Convenção sem  $(1 - \beta)$ : PyTorch e o paper original de Polyak escrevem  $v_{t+1} = \beta v_t + \nabla_{\theta} \mathcal{L}$ . A diferença é absorvível em  $\eta$ .

---

<sup>3</sup>Boris T. Polyak. "Some methods of speeding up the convergence of iteration methods". Em: [USSR Computational Mathematics and Mathematical Physics](#) 4.5 (1964), pp. 1–17.

## Efeito visual do *Momentum*



- Esquerda: SGD puro oscila fortemente em vales estreitos.
- Direita: com momentum, oscilações verticais se cancelam e a trajetória avança mais rápido na direção principal.



## Nesterov Accelerated Momentum<sup>4</sup>

---

- Momentum é, na prática, um *preditor* do próximo passo.
- Nesterov sugere avaliar o gradiente **já no ponto deslocado pelo momentum**, e não no ponto atual.

### Regra de Nesterov

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla_{\theta} \mathcal{L}(\theta_t - \eta v_t)$$

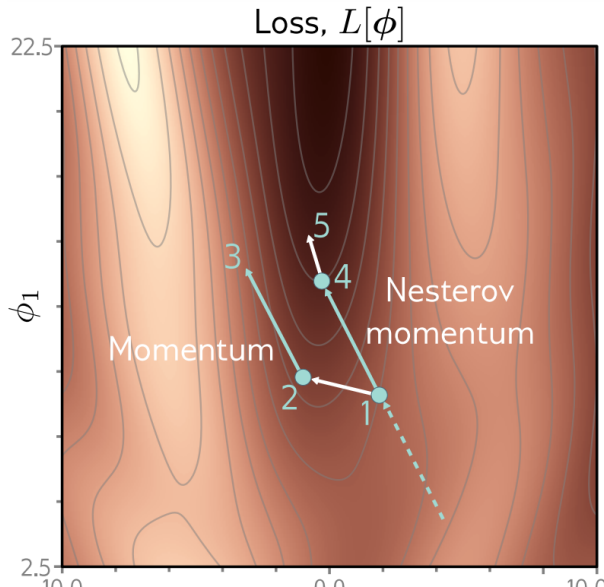
$$\theta_{t+1} = \theta_t - \eta v_{t+1}$$

- Pequena correção, ganho prático em problemas com vales alongados.

---

<sup>4</sup>Yurii Nesterov. “A method of solving a convex programming problem with convergence rate  $O(1/k^2)$ ”. Em: [Soviet Mathematics Doklady](#) 27.2 (1983), pp. 372–376.

## Nesterov: olhar à frente antes de calcular o gradiente



- O ponto branco é onde o momentum levaria, e é nesse ponto que o gradiente é avaliado.
- Resultado: a trajetória do Nesterov “corrige” a do momentum quando há curvatura.

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
5. Momentum e Nesterov
- 6. Métodos com Taxa Adaptativa**
7. MUON: a próxima geração
8. Encerramento

## Problema do passo fixo em paisagens anisotrópicas

---

- Em uma direção a loss pode ser muito íngreme; em outra, quase plana.
- Com  $\eta$  fixo, ou damos passos exagerados na direção íngreme, ou ficamos lentos na direção rasa.
- Solução: usar uma taxa de aprendizado **efetiva** diferente para cada parâmetro, baseada na magnitude histórica de seus gradientes.

## Normalização do gradiente

---

Para tornar o passo independente da magnitude do gradiente, podemos definir

$$m_{t+1} = \nabla_{\theta} \mathcal{L}(\theta_t), \quad v_{t+1} = (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

e atualizar com

$$\theta_{t+1} = \theta_t - \eta \frac{m_{t+1}}{\sqrt{v_{t+1}} + \epsilon}.$$

- Com a normalização, o termo que multiplica  $\eta$  tem magnitude próxima de 1.
- O tamanho do passo passa a ser controlado essencialmente por  $\eta$ .
- $\epsilon$  é um pequeno valor positivo para estabilidade numérica.

# AdaGrad<sup>5</sup>

## Atualização

$$G_{t+1} = G_t + (\nabla_{\theta} \mathcal{L})^2$$
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_{t+1}} + \epsilon} \nabla_{\theta} \mathcal{L}$$

- *Adaptive (per-parameter) Gradient.*
- Cada parâmetro tem sua própria taxa de aprendizado efetiva, baseada em  $G_t$ , a soma acumulada dos gradientes ao quadrado.
- Parâmetros com gradientes consistentemente grandes recebem passos menores; parâmetros raros recebem passos maiores.

## Limitação

$G_t$  é monotonamente crescente, logo  $\frac{\eta}{\sqrt{G_t} + \epsilon}$  decai sempre. Após muitas iterações, a taxa efetiva tende a zero e o aprendizado para, mesmo se ainda houver progresso a fazer. Como evitar a decadência sem perder a normalização por parâmetro?

# RMSPProp<sup>6</sup>

- *Root Mean Square Propagation.*
- Apresentado por Hinton em um *slide* de aula; nunca foi formalmente publicado.
- Disponível em praticamente todos os *frameworks*.
- Ideia: substituir a soma do AdaGrad por uma **média móvel exponencial**.

## Atualização

$$\begin{aligned}\mathbb{E}[g^2]_{t+1} &= \rho \mathbb{E}[g^2]_t + (1 - \rho) (\nabla_{\theta} \mathcal{L})^2 \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\mathbb{E}[g^2]_{t+1} + \epsilon}} \nabla_{\theta} \mathcal{L}\end{aligned}$$

- $\rho$  controla a janela efetiva de memória; o termo deixa de crescer indefinidamente.

<sup>6</sup>Tijmen Tieleman e Geoffrey Hinton. Lecture 6.5—RMSPProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning. 2012.

## Adam<sup>7</sup>: combinando momentum e normalização

---

- *Adaptive Moment Estimation.*
- Junta as duas ideias vistas: **momentum** (sobre o gradiente) e **normalização** (pela raiz da média móvel do gradiente ao quadrado).
- Considerado, por muitos anos, o otimizador padrão para deep learning.

---

<sup>7</sup>Diederik P. Kingma e Jimmy Ba. “Adam: A Method for Stochastic Optimization”. Em: [ICLR](#). 2015.



## Adam: as duas médias móveis

---

### Estimativas dos momentos

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta_t)$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta_t))^2$$

- $\beta_1 \in [0, 1)$ ,  $\beta_2 \in [0, 1)$ , com  $m_0 = 0$  e  $v_0 = 0$ .
- $m$  acompanha o primeiro momento (média) do gradiente,  $v$  acompanha o segundo momento (média do gradiente ao quadrado).

## Adam: correção de viés inicial

---

- Com  $m_0 = v_0 = 0$ , as primeiras iterações subestimam fortemente  $m$  e  $v$ .
- Correção de viés (*bias correction*):

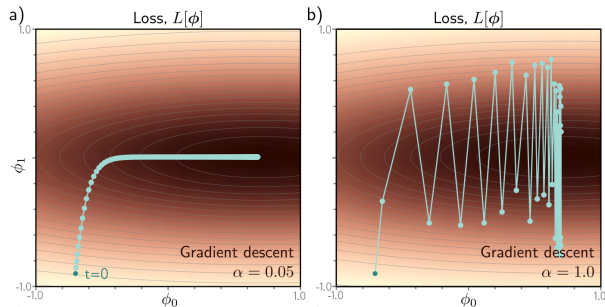
$$\tilde{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}, \quad \tilde{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}.$$

- A regra final é

$$\theta_{t+1} = \theta_t - \eta \frac{\tilde{m}_{t+1}}{\sqrt{\tilde{v}_{t+1}} + \epsilon}$$

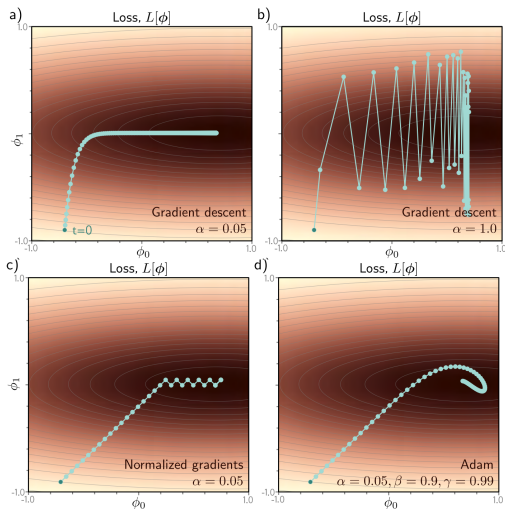
- Hiperparâmetros padrão:  $\beta_1=0.9$ ,  $\beta_2=0.999$ ,  $\epsilon=10^{-8}$ .

# Por que o Adam ajuda em loss anisotrópica



- Esquerda:  $\eta$  pequeno avança bem na vertical, mas é lento na horizontal.
- Direita:  $\eta$  grande progride na horizontal, mas oscila descontroladamente na vertical.
- Adam normaliza por direção, evitando ambos os extremos.

# Adam comparado com outros otimizadores



## Por que Adam virou o padrão

---

- As magnitudes dos gradientes variam fortemente conforme a função de ativação, a camada da rede e a inicialização.
- Adam é mais imune a essas diferenças e menos sensível à escolha da taxa de aprendizado inicial.
- Na prática, costuma funcionar razoavelmente bem com os hiperparâmetros padrão.

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
- 7. MUON: a próxima geração**
8. Encerramento

# O que mudou desde Adam (2015)?

---

- Por quase uma década, Adam (e variantes como AdamW) foi o padrão de fato em deep learning.
- Em 2024, o otimizador **MUON**<sup>8</sup> foi proposto como o primeiro avanço fundamental significativo desde Adam.
- Foco do MUON: as **matrizes de pesos** das camadas ocultas (parâmetros 2D). Bias e *embeddings* continuam usando Adam ou SGD.

---

<sup>8</sup>Keller Jordan et al. Muon: An optimizer for hidden layers in neural networks. 2024. URL: <https://kellerjordan.github.io/posts/muon/>.

## Ideia central: ortogonalizar a atualização

---

- As matrizes de gradientes em redes neurais costumam ter *rank efetivo* baixo, ou seja, são dominadas por poucas direções singulares.
- Isso desperdiça capacidade do passo: direções pequenas mas úteis recebem magnitude desprezível.
- MUON substitui a matriz de atualização pela **matriz ortogonal mais próxima**, redistribuindo a magnitude entre todas as direções singulares.
- Geometricamente, o passo do MUON é a *steepest descent* sob a norma espectral, não sob a norma de Frobenius.



# Pseudocódigo do MUON

---

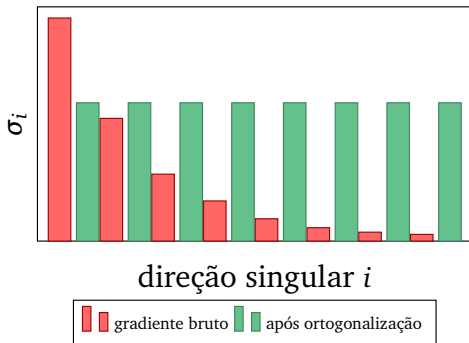
## Atualização (matriz $G$ do gradiente)

- 1:  $M_{t+1} \leftarrow \beta M_t + (1 - \beta) G_t$  ▷ momentum SGD com Nesterov
- 2:  $\widehat{M}_{t+1} \leftarrow M_{t+1} / \|M_{t+1}\|_F$  ▷ normaliza pela norma de Frobenius
- 3:  $O_{t+1} \leftarrow \text{NewtonSchulz5}(\widehat{M}_{t+1})$  ▷ aproxima a matriz ortogonal mais próxima
- 4:  $W_{t+1} \leftarrow W_t - \eta O_{t+1}$

- Newton-Schulz é uma iteração de polinômios matriciais que aproxima a parte ortogonal de  $\widehat{M}$  sem calcular SVD.
- Com 5 passos, roda eficientemente em *tensor cores* de GPU.

## Intuição: por que ortogonalizar ajuda?

- Gradiente típico: poucas direções com magnitude muito alta, muitas direções com magnitude pequena.
- Adam normaliza *por parâmetro* (entrada da matriz), mas não trata o problema de *rank* efetivo.
- MUON normaliza *por direção singular*: todas as direções recebem magnitude semelhante.
- Resultado: melhor uso da capacidade de cada passo, especialmente em redes largas.



## Resultados empíricos

---

- Recordes de velocidade em *NanoGPT speedrun* ( $\sim 1.35\times$  speedup para atingir loss de validação 3.28).
- Recordes em CIFAR-10 *speedrun* (de 3.3 para 2.6 segundos-A100 para atingir 94% de acurácia).
- Escalabilidade demonstrada até 1.5B de parâmetros (Liu et al., 2025).

### Limitações práticas

Aplica-se apenas a parâmetros 2D (matrizes). Vetores (*biases*, *layer norm*) e *embeddings* continuam sendo otimizados com AdamW.

# Visão Geral

---

1. Por que não força bruta
2. Derivando a Descida de Gradiente
3. Convexidade e Paisagem da Loss
4. Tamanho do Mini-batch
5. Momentum e Nesterov
6. Métodos com Taxa Adaptativa
7. MUON: a próxima geração
- 8. Encerramento**

## Qual otimizador usar na prática?

---

- **Padrão hoje:** AdamW (Adam com *weight decay* desacoplado) na maioria dos modelos.
- **Visão computacional clássica:** SGD com momentum costuma generalizar tão bem ou melhor, especialmente com *learning rate schedules* bem ajustados.
- **Modelos grandes (LLMs):** Adam/AdamW continuam sendo o padrão; MUON é emergente e relevante para parâmetros 2D em camadas ocultas.
- **Importante:** a taxa de aprendizado  $\eta$  é o hiperparâmetro de maior impacto em qualquer um deles.

# Leituras Recomendadas

---

- Capítulos 4 e 8 de Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>
- Capítulo 6 de Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023
- Adam original: Diederik P. Kingma e Jimmy Ba. “Adam: A Method for Stochastic Optimization”. Em: ICLR. 2015
- MUON: Keller Jordan et al. Muon: An optimizer for hidden layers in neural networks. 2024. URL: <https://kellerjordan.github.io/posts/muon/>

# Referências I

---

- [1] John Duchi, Elad Hazan e Yoram Singer. “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization”. Em: JMLR 12 (2011), pp. 2121–2159.
- [2] Ian Goodfellow, Yoshua Bengio e Aaron Courville. Deep Learning. MIT Press, 2016. URL: <https://www.deeplearningbook.org/>.
- [3] Keller Jordan et al. Muon: An optimizer for hidden layers in neural networks. 2024. URL: <https://kellerjordan.github.io/posts/muon/>.
- [4] Diederik P. Kingma e Jimmy Ba. “Adam: A Method for Stochastic Optimization”. Em: ICLR. 2015.
- [5] Yurii Nesterov. “A method of solving a convex programming problem with convergence rate  $O(1/k^2)$ ”. Em: Soviet Mathematics Doklady 27.2 (1983), pp. 372–376.
- [6] Boris T. Polyak. “Some methods of speeding up the convergence of iteration methods”. Em: USSR Computational Mathematics and Mathematical Physics 4.5 (1964), pp. 1–17.
- [7] Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023.
- [8] Tijmen Tieleman e Geoffrey Hinton. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning. 2012.